

Практическая работа 7: Поведение при нажатии кнопки

В этом разделе вы узнали, как добавить кнопку в приложение и как изменить приложение, чтобы оно реагировало на нажатие кнопки. Теперь пришло время применить полученные знания на практике, создав приложение.

Вы создадите приложение под названием Lemonade app. Сначала прочитайте требования к приложению Lemonade, чтобы понять, как оно должно выглядеть и вести себя. Если вы хотите испытать себя, вы можете создать приложение самостоятельно. Если вы застряли, читайте последующие разделы, чтобы получить больше подсказок и советов о том, как разбить проблему на части и решить ее шаг за шагом.

Проработайте эту практическую задачу в удобном для вас темпе. Уделите столько времени, сколько вам нужно для создания каждой части функциональности приложения. Код решения для приложения Lemonade доступен в конце, но мы рекомендуем вам попробовать собрать приложение самостоятельно, прежде чем проверять решение. Помните, что представленное решение не является единственным способом создания приложения Lemonade, поэтому вы можете создать его по-другому, если требования к приложению соблюдены.

Необходимые условия

- Уметь создать простой макет пользовательского интерфейса в Compose с композитными элементами текста и изображений
- Уметь создавать интерактивное приложение, реагирующее на нажатие кнопки
- Базовое понимание композиции и рекомпозиции
- Знакомство с основами языка программирования Kotlin, включая функции, переменные, условия и лямбды

Что вам понадобится

Компьютер с доступом в Интернет и установленной программой Android Studio.

2. Обзор приложения

Вы поможете нам воплотить в жизнь нашу идею создания цифрового лимонада! Цель - создать простое интерактивное приложение, которое позволит вам выжимать сок из лимонов, касаясь изображения на экране, пока у вас не получится стакан лимонада. Считайте это метафорой или просто забавным способом скоротать время!

Вот как работает приложение:

Когда пользователь впервые запускает приложение, он видит лимонное дерево. Там есть ярлык, который предлагает коснуться изображения лимонного дерева, чтобы «выбрать» лимон с дерева. После нажатия на лимонное дерево пользователь видит лимон. Ему предлагается коснуться лимона,

чтобы «выжать» его и приготовить лимонад. Чтобы сжать лимон, нужно коснуться его несколько раз. Количество нажатий, необходимое для выжимания лимона, каждый раз разное и представляет собой случайно сгенерированное число от 2 до 4 (включительно). После того как они коснутся лимона необходимое количество раз, они увидят освежающий стакан лимонада! Им предлагается коснуться стакана, чтобы «выпить» лимонад. После того как они коснутся стакана с лимонадом, они увидят пустой стакан. Их просят коснуться пустого стакана, чтобы начать сначала. После того как они коснутся пустого стакана, они увидят лимонное дерево и смогут начать процесс заново. Больше лимонада, пожалуйста! Вот более крупные скриншоты того, как выглядит приложение:

Для каждого этапа приготовления лимонада на экране есть свое изображение и текстовый ярлык, а также разное поведение приложения в ответ на нажатие. Например, когда пользователь нажимает на лимонное дерево, приложение показывает лимон.

Ваша задача - создать макет пользовательского интерфейса приложения и реализовать логику, позволяющую пользователю пройти все этапы приготовления лимонада.

3. Начните работу

Создайте проект В Android Studio создайте новый проект с шаблоном Empty Activity со следующими данными:

Имя: Lemonade Название пакета: com.example.lemonade Минимальный SDK: 24 Когда приложение успешно создано и проект собран, переходите к следующему разделу.

Добавление изображений Вам предоставляются четыре векторных файла для рисования, которые вы будете использовать в приложении Lemonade.

Получите эти файлы:

Скачайте zip-файл с изображениями для приложения. Дважды щелкните по zip-файлу. Этот шаг распакует изображения в папку. Добавьте изображения в папку drawable вашего приложения. Если вы не помните, как это сделать, обратитесь к кодебалету «Создание интерактивного приложения Dice Roller».

Папка вашего проекта должна выглядеть как на следующем снимке экрана, где активы lemon_drink.xml, lemon_restart.xml, lemon_squeeze.xml и lemon_tree.xml теперь находятся в каталоге res > drawable:

Дважды щелкните файл с векторным рисунком, чтобы увидеть предварительный просмотр изображения. Выберите панель Design (не вид Code или Split), чтобы увидеть изображение во всю ширину.

После того как файлы изображений включены в приложение, вы можете ссылаться на них в коде. Например, если файл векторного отрисовщика называется lemon_tree.xml, то в коде на Kotlin вы можете ссылаться на отрисовщик, используя его идентификатор ресурса в формате R.drawable.lemon_tree.

Добавьте строковые ресурсы Добавьте следующие строки в проект в файл res > values > strings.xml:

Нажмите на лимонное дерево, чтобы выбрать лимон Продолжайте касаться лимона, чтобы сжать его Коснитесь лимонада, чтобы выпить его Коснитесь пустого стакана, чтобы начать сначала Следующие строки также необходимы в вашем проекте. Они не отображаются на экране в пользовательском интерфейсе, но они используются в описании содержимого изображений в вашем приложении, чтобы описать, что это за изображения. Добавьте эти дополнительные строки в файл `strings.xml` вашего приложения:

Лимонное дерево Лимон Стакан с лимонадом Пустой стакан Если вы не помните, как объявлять строковые ресурсы в своем приложении, посмотрите [codelab по созданию интерактивного приложения Dice Roller](#) или обратитесь к `String`. Дайте каждому строковому ресурсу соответствующее имя-идентификатор, описывающее значение, которое он содержит. Например, для строки «Lemon» вы можете объявить ее в файле `strings.xml` с именем-идентификатором `lemon_content_description`, а затем ссылаться на нее в коде, используя идентификатор ресурса: `R.string.lemon_content_description`.

Шаги по созданию лимонада Теперь у вас есть строковые ресурсы и графические активы, необходимые для реализации приложения. Вот краткое описание каждого шага приложения и того, что отображается на экране:

Шаг 1:

Текст: Нажмите на лимонное дерево, чтобы выбрать лимон Изображение: Лимонное дерево (`lemon_tree.xml`)

Шаг 2:

Текст: Продолжайте касаться лимона, чтобы выжать его Изображение: Лимон (`lemon_squeeze.xml`)

Шаг 3:

Текст: Коснитесь лимонада, чтобы выпить его Изображение: Полный стакан лимонада (`lemon_drink.xml`)

Шаг 4:

Текст: Нажмите на пустой стакан, чтобы начать сначала Изображение: Пустой стакан (`lemon_restart.xml`)

Добавьте визуальной полировки Чтобы ваша версия приложения выглядела так же, как эти финальные скриншоты, нужно сделать еще несколько визуальных настроек в приложении:

Увеличьте размер шрифта текста, чтобы он был больше, чем размер шрифта по умолчанию (например, 18sp). Добавьте дополнительное пространство между текстовой надписью и изображением под ней, чтобы они не находились слишком близко друг к другу (например, 16dp). Придайте кнопке акцентный цвет и слегка закругленные углы, чтобы дать пользователям понять, что они могут нажимать на изображение. Если вы хотите испытать себя, создайте остальную часть приложения, основываясь на

описании того, как оно должно работать. Если вам нужны дополнительные рекомендации, переходите к следующему разделу.

Предупреждение: Не продолжайте читать остальные инструкции, если не хотите получить подсказки. Рекомендуется попробовать решить проблему самостоятельно и обращаться к остальным инструкциям только в том случае, если вы застряли.

4. Планируйте, как создать приложение

При создании приложения лучше всего сначала сделать минимальную рабочую версию приложения. Затем постепенно добавляйте функциональность, пока не завершите все желаемые функции. Определите небольшой фрагмент сквозной функциональности, который вы можете построить первым.

В приложении Lemonade обратите внимание, что ключевой частью приложения является переход от одного шага к другому, при этом каждый раз показывается другое изображение и текстовая надпись. Изначально вы можете игнорировать особое поведение состояния сжатия, потому что вы сможете добавить эту функциональность позже, когда построите основу приложения.

Ниже приведен обзор шагов, которые вы можете предпринять для создания приложения:

Создайте макет пользовательского интерфейса для первого шага приготовления лимонада, который предлагает пользователю выбрать лимон с дерева. Границу вокруг изображения можно пока пропустить, потому что это визуальная деталь, которую можно добавить позже.

Реализуйте поведение в приложении, чтобы при нажатии на лимонное дерево оно показывало изображение лимона и соответствующую текстовую надпись. Это охватывает первые два этапа приготовления лимонада.

Добавьте код, чтобы при каждом нажатии на изображение приложение отображало остальные шаги по приготовлению лимонада. В этот момент однократное нажатие на лимон может перейти к отображению стакана с лимонадом.

Добавьте пользовательское поведение для этапа выжимания лимона, чтобы пользователь должен был «выжать», или коснуться, лимона определенное количество раз, которое генерируется случайным образом от 2 до 4.

Завершите работу над приложением, добавив все необходимые детали визуальной полировки. Например, измените размер шрифта или добавьте рамку вокруг изображения, чтобы приложение выглядело более полированным. Убедитесь в том, что приложение следует хорошим практикам кодирования, например, придерживается руководства по стилю кодирования Kotlin и добавляет комментарии к коду.

Если вы можете использовать эти основные шаги для реализации приложения Lemonade, приступайте к его созданию самостоятельно. Если же вы обнаружите, что вам нужны дополнительные рекомендации по каждому из этих пяти шагов, переходите к следующему разделу.

5. Реализуйте приложение

Создайте макет пользовательского интерфейса. Сначала измените приложение так, чтобы в центре экрана отображалось изображение лимонного дерева и соответствующая ему текстовая надпись, которая гласит: «Нажмите на лимонное дерево, чтобы выбрать лимон». Между текстом и изображением под ним должно быть 16dp пространства.

Если это поможет, вы можете использовать следующий стартовый код в файле MainActivity.kt:

```
package com.example.lemonade

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.lemonade.ui.theme.LemonadeTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            LemonadeTheme {
                LemonApp()
            }
        }
    }
}

@Composable
fun LemonApp() {
    // A surface container using the 'background' color from the theme
    Surface(
        modifier = Modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Text(text = "Hello there!")
    }
}
```

```
@Preview(showBackground = true)
@Composable
fun DefaultPreview() {
    LemonadeTheme {
        LemonApp()
    }
}
```

Этот код похож на код, который автоматически генерируется Android Studio. Однако вместо композита `Greeting()` определен композит `LemonApp()`, который не ожидает параметра. Композит `DefaultPreview()` также обновлен для использования композита `LemonApp()`, чтобы вы могли легко просматривать свой код.

После ввода этого кода в Android Studio измените компонент `LemonApp()`, который должен содержать содержимое приложения. Вот несколько вопросов, которые помогут вам сориентироваться в процессе работы:

Какие компоненты вы будете использовать? Существует ли стандартный компонент компоновки Compose, который поможет вам расположить компонент в нужных местах?

Важно: чтобы сделать ваше приложение доступным для большего числа пользователей, не забудьте установить описания содержимого для изображений, чтобы описать, что содержит изображение.

Выполните этот шаг, чтобы при запуске приложения в нем отображались лимонное дерево и текстовая метка. Предварительно просмотрите ваше приложение в Android Studio, чтобы увидеть, как выглядит пользовательский интерфейс при изменении кода. Запустите приложение, чтобы убедиться, что оно выглядит так же, как на скриншоте, который вы видели ранее в этом разделе.

Вернитесь к этим инструкциям, если вам нужны дополнительные указания по добавлению поведения при нажатии на изображение.

Добавление поведения при нажатии Далее вы добавите код, чтобы при нажатии на изображение лимонного дерева появлялось изображение лимона вместе с текстовой надписью `Keep tapping the lemon to squeeze it`. Другими словами, когда вы касаетесь лимонного дерева, текст и изображение меняются.

Ранее в этом курсе вы узнали, как сделать кнопку кликабельной. В случае с приложением `Lemonade` композитная кнопка отсутствует. Однако вы можете сделать любую композицию, а не только кнопку, кликабельной, если укажете для нее модификатор `clickable`. Пример смотрите на странице документации по `clickable`.

Что должно происходить при нажатии на изображение? Код для реализации этого поведения нетривиален, поэтому сделайте шаг назад и посмотрите на знакомое приложение.

Посмотрите на приложение `Dice Roller` Пересмотрите код приложения `Dice Roller`, чтобы увидеть, как оно отображает различные изображения костей в зависимости от значения броска костей:

MainActivity.kt in Dice Roller app

```

...

@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf(1) }
    val imageResource = when(result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontally)
    {
        Image(painter = painterResource(id = imageResource), contentDescription =
result.toString())
        Button(onClick = { result = (1..6).random() }) {
            Text(stringResource(id = R.string.roll))
        }
    }
}

...

```

Ответьте на эти вопросы о коде приложения Dice Roller:

Значение какой переменной определяет соответствующее изображение игральных костей? Какое действие пользователя приводит к изменению этой переменной? Составная функция `DiceWithButtonAndImage()` сохраняет последний бросок костей в переменной `result`, которая была определена с помощью составной функции `remember` и функции `mutableStateOf()` в этой строке кода:

```
var result by remember { mutableStateOf(1) }
```

Когда переменная `result` обновляется до нового значения, Compose запускает рекомпозицию композита `DiceWithButtonAndImage()`, что означает, что композит будет выполнен снова. Значение результата запоминается при всех повторных композициях, поэтому при повторном выполнении композита `DiceWithButtonAndImage()` будет использоваться последнее значение результата. Используя оператор `when` для значения переменной `result`, композит определяет новый идентификатор ресурса `drawable` для показа, и композит `Image` отображает его.

Примените полученные знания к приложению Lemonade Теперь ответьте на аналогичные вопросы о приложении Lemonade:

Есть ли переменная, которую можно использовать для определения того, какой текст и изображение должны быть показаны на экране? Определите эту переменную в своем коде. Можете ли вы использовать условные выражения в Kotlin, чтобы приложение выполняло различные действия в зависимости от значения этой переменной? Если да, напишите условный оператор в своем коде. Какое

действие пользователя вызывает изменение этой переменной? Найдите в коде подходящее место, где это происходит. Добавьте туда код для обновления переменной.

Примечание: Возможно, вы захотите обозначить каждый шаг приготовления лимонада цифрой. Например, на шаге 1 приложения есть изображение лимонного дерева, а щелчок по изображению переводит на шаг 2 приложения. Это поможет вам организовать, какая текстовая строка соответствует какому изображению. https://developer.android.com/static/codelabs/basic-android-kotlin-compose-button-click-practice-problem/img/270ecd406fc30120_856.png

Этот раздел может быть довольно сложным в реализации и требует изменений во многих местах вашего кода для корректной работы. Не расстраивайтесь, если приложение сразу не будет работать так, как вы ожидаете. Помните, что существует несколько правильных способов реализовать это поведение.

Когда вы закончите, запустите приложение и убедитесь, что оно работает. Когда вы запустите приложение, оно должно показать изображение лимонного дерева и соответствующую текстовую надпись. Одно нажатие на изображение лимонного дерева должно обновить текстовую надпись и показать изображение лимона. Пока что нажатие на изображение лимона не должно ничего делать.

Добавьте оставшиеся шаги Теперь ваше приложение может отображать два шага по приготовлению лимонада! На данный момент ваш компонент `LemonApp()` может выглядеть примерно так, как показано в следующем фрагменте кода. Ничего страшного, если ваш код будет выглядеть немного иначе, главное, чтобы поведение приложения было одинаковым.

```
...
@Composable
fun LemonApp() {
    // Current step the app is displaying (remember allows the state to be retained
    // across recompositions).
    var currentStep by remember { mutableStateOf(1) }

    // A surface container using the 'background' color from the theme
    Surface(
        modifier = Modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        when (currentStep) {
            1 -> {
                Column (
                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.Center,
                    modifier = Modifier.fillMaxSize()
                ){
                    Text(text = stringResource(R.string.lemon_select))
                    Spacer(modifier = Modifier.height(32.dp))
                    Image(
                        painter = painterResource(R.drawable.lemon_tree),
                        contentDescription =
                            stringResource(R.string.lemon_tree_content_description),
                        modifier = Modifier
                            .wrapContentSize()
                            .clickable {
```



```

                                currentStep = 2
                                }
                            )
                        }
                    }
                2 -> {
                    Column (
                        horizontalAlignment = Alignment.CenterHorizontally,
                        verticalArrangement = Arrangement.Center,
                        modifier = Modifier.fillMaxSize()
                    ){
                        Text(text = stringResource(R.string.lemon_squeeze))
                        Spacer(modifier = Modifier.height(32
                            .dp))
                        Image(
                            painter = painterResource(R.drawable.lemon_squeeze),
                            contentDescription =
                                stringResource(R.string.lemon_content_description),
                            modifier = Modifier.wrapContentSize()
                        )
                    }
                }
            }
        }
    }
    ...

```

Затем вы добавите остальные шаги по приготовлению лимонада. Одно нажатие на изображение должно перевести пользователя на следующий шаг приготовления лимонада, где обновляются и текст, и изображение. Вам нужно будет изменить код, чтобы сделать его более гибким для обработки всех шагов приложения, а не только первых двух.

Чтобы при каждом нажатии на изображение оно вело себя по-разному, необходимо настроить поведение клика. Если говорить точнее, то лямбда, которая выполняется при нажатии на изображение, должна знать, к какому шагу мы переходим.

Вы можете заметить, что в вашем приложении повторяется код для каждого шага создания лимонада. Для оператора `when` в предыдущем фрагменте кода код для случая 1 очень похож на код для случая 2 с небольшими отличиями. Если это полезно, создайте новую составную функцию, например, `LemonTextAndImage()`, которая отображает текст над изображением в пользовательском интерфейсе. Создав новую составную функцию, которая принимает некоторые входные параметры, вы получаете многократно используемую функцию, которая будет полезна в различных сценариях, если вы измените входные параметры, которые вы передаете. Ваша задача - выяснить, какими должны быть входные параметры. После того как вы создадите эту композитную функцию, обновите существующий код, чтобы вызвать новую функцию в соответствующих местах.

Еще одно преимущество наличия отдельной композиции, такой как `LemonTextAndImage()`, заключается в том, что ваш код становится более организованным и надежным. Когда вы вызываете `LemonTextAndImage()`, вы можете быть уверены, что и текст, и изображение будут обновлены до новых

значений. В противном случае легко случайно пропустить случай, когда обновленная текстовая метка отображается с неправильным изображением.

Вот еще одна подсказка: вы даже можете передать в composable лямбда-функцию. Обязательно используйте нотацию типа функции, чтобы указать, какой тип функции следует передать. В следующем примере определена композитная функция WelcomeScreen(), которая принимает два входных параметра: строку имени и функцию onStartClicked() типа () -> Unit. Это означает, что функция не принимает входных параметров (пустые круглые скобки перед стрелкой) и не имеет возвращаемого значения (единица после стрелки). Любая функция, соответствующая типу функции () -> Unit, может быть использована для установки обработчика onClick этой кнопки. Когда кнопка нажимается, вызывается функция onStartClicked().

```
@Composable
fun WelcomeScreen(name: String, onStartClicked: () -> Unit) {
    Column {
        Text(text = "Welcome $name!")
        Button(
            onClick = onStartClicked
        ){
            Text("Start")
        }
    }
}
```

Передача лямбды в составной элемент - полезный паттерн, потому что в этом случае составной элемент WelcomeScreen() можно повторно использовать в различных сценариях. Имя пользователя и поведение кнопки onClick могут быть разными каждый раз, потому что они передаются в качестве аргументов.

С этими дополнительными знаниями вернитесь к своему коду, чтобы добавить оставшиеся шаги по созданию лимонада в ваше приложение.

Вернитесь к этим инструкциям, если вам нужны дополнительные указания по добавлению пользовательской логики, связанной с выжиманием лимона случайное количество раз.

Добавьте логику выдавливания

Отличная работа! Теперь у вас есть основа приложения. Нажатие на изображение должно переводить вас с одного шага на другой. Пришло время добавить поведение, при котором нужно выжать лимон несколько раз, чтобы сделать лимонад. Количество раз, которое пользователь должен выжать или коснуться лимона, должно быть случайным числом от 2 до 4 (включительно). Это случайное число будет меняться каждый раз, когда пользователь берет новый лимон с дерева.

Вот несколько вопросов, которые помогут вам сориентироваться:

Как генерировать случайные числа в Kotlin? В какой точке кода вы должны генерировать случайное число? Как убедиться, что пользователь коснулся лимона необходимое количество раз, прежде чем

перейти к следующему шагу? Нужны ли вам какие-либо переменные, хранящиеся в `remember composable`, чтобы данные не сбрасывались каждый раз, когда экран перерисовывается? Когда вы закончите внедрение этих изменений, запустите приложение. Убедитесь, что для перехода к следующему шагу требуется несколько касаний изображения лимона и что количество касаний каждый раз составляет случайное число от 2 до 4. Если при одном нажатии на изображение лимона на экране появляется стакан с лимонадом, вернитесь к коду, чтобы понять, чего не хватает, и попробуйте еще раз.

Вернитесь к этим инструкциям, если вам нужны дополнительные указания по завершению работы над приложением.

Завершение работы над приложением Вы почти закончили! Добавьте несколько последних деталей, чтобы отполировать приложение.

Напоминаем, что здесь представлены финальные скриншоты того, как выглядит приложение:

Выровняйте текст и изображения по вертикали и горизонтали в пределах экрана. Установите размер шрифта текста на 18sp. Добавьте 16 dp пространства между текстом и изображением. Добавьте тонкую рамку размером 2dp вокруг изображений со слегка закругленными углами размером 4dp. Граница имеет значение цвета RGB: 105 для красного, 205 для зеленого и 216 для синего. Примеры того, как добавить границу, вы можете найти в Google. Также вы можете обратиться к документации по `Border`. После завершения этих изменений запустите приложение и сравните его с финальными скриншотами, чтобы убедиться, что они совпадают.

Как часть хорошей практики кодирования, вернитесь назад и добавьте комментарии к коду, чтобы любому, кто прочитает ваш код, было легче понять ход ваших мыслей. Удалите все операторы импорта в верхней части файла, которые не используются в вашем коде. Убедитесь, что ваш код следует руководству по стилю Kotlin. Все эти усилия помогут сделать ваш код более читаемым для других людей и более простым в сопровождении!

Молодцы! Вы проделали потрясающую работу по созданию приложения Lemonade! Это было сложное приложение, в котором нужно было разобраться со многими деталями. Теперь побалуйте себя бокалом освежающего лимонада. За ваше здоровье!